

プログラミング レポート 2

202512228 小松侑輝

2026 年 7 月

1 はじめに

本レポートでは、標準入力から得た文字列を動的に確保したメモリへ保存するプログラムと、保存した文字列の一部を別の文字列へ置換するプログラムを作成した。実行時に必要な大きさのメモリを得るには `malloc` を用い、使用後には `free` で解放する必要がある。

課題の指示に従い、大域変数を使用しなかった。入力には `fgets` を用い、文字列の長さの取得には `strlen`、文字列のコピーには `strcpy` 部分文字列の探索には `strstr` を用いた。動的メモリを確保する際には、文字列末尾のヌル文字の領域を含めるため、文字列長に 1 を加えた大きさを指定した。さらに、`malloc` の返り値が `NULL` でないことを必ず確認した。

2 設問 1

2.1 設問内容の理解と実装方針

設問 1 では、動的に確保したメモリ上の文字列を最大 100 個保存し、入力終了後に 3 桁の行番号を付けて表示する。そこで、文字列そのものを固定長の二次元配列へ保存するのではなく、文字列の先頭アドレスを格納する `char*` 型の配列 `lines` を用意する。入力時には一時的な文字配列 `buf` に `fgets` で 1 行を読み込む。読み込んだ文字列の先頭 3 文字が ... であれば保存せずに入力を終了する。

通常の文字列であれば、`strlen(buf)+1` バイトを `malloc` で確保し、`strcpy` で内容をコピーする。この方法では、各行について実際に必要な最小限のメモリだけを確保できる。入力終了後は、書式指定 `%03d` を用いて行番号を常に 3 桁で表示し、最後に確保したすべての領域を `free` で解放する。

2.2 作成したプログラムとポイント

作成したプログラムをリスト 1 に示す。

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define LINES 100
6 #define BSIZE 100
7
8 int main(void){
9     char *lines[LINES];
10    char buf[BSIZE];
11    char *p;
```

```

12     int n = 0;
13     int i;
14     size_t len;
15
16     while(n < LINES){
17         if(fgets(buf, BSIZE, stdin) == NULL){
18             break;
19         }
20
21         if(buf[0] == '.' && buf[1] == '.' && buf[2] == '.'){
22             break;
23         }
24
25         p = (char*)malloc(strlen(buf) + 1);
26         if(p == NULL){
27             printf("memory allocation failure.\n");
28             for(i = 0; i < n; i++){
29                 free(lines[i]);
30             }
31             exit(EXIT_FAILURE);
32         }
33
34         strcpy(p, buf);
35         lines[n] = p;
36         n++;
37     }
38
39     for(i = 0; i < n; i++){
40         printf("%03d %s", i + 1, lines[i]);
41         len = strlen(lines[i]);
42         if(len == 0 || lines[i][len - 1] != '\n'){
43             printf("\n");
44         }
45     }
46
47     for(i = 0; i < n; i++){
48         free(lines[i]);
49     }
50
51     return 0;
52 }

```

Listing 1 設問1のプログラム (s2512228-1.c)

リスト1の9行目では、100個の文字列のアドレスを保存する配列を宣言している。16行目の繰り返し条件を `n < LINES` とすることで、配列の範囲外へ書き込まないようにした。21行目では短絡評価を利用し、先頭から3文字がすべてピリオドである場合に入力を終了している。

25行目では、読み込んだ文字列の長さにヌル文字の1バイトを加えた大きさを確保している。26–32行目では `malloc` の失敗を確認し、失敗した場合にはそれまでに確保した領域を解放してから異常終了する。34–36行目で文字列を確保領域へコピーし、その先頭アドレスを `lines` に保存する。39–45行目では行番号を3桁で表示する。通常は入力文字列に改行コードが含まれるが、バッファの上限まで入力された場合にも次の行と連結しないよう、末尾に改行がない場合は追加で改行を出力するようにした。47–49行目では、全行の表示後に確保領域を解放している。

2.3 確認方法と結果

コンパイルは次のコマンドで行った。

```
gcc -std=c89 -o s2512228-1.exe s2512228-1.c
```

作成したプログラムに対して、alpha、beta、gamma の 3 行を入力し、その後に終了条件である「...」を入力した。実行結果を図 1 に示す。

```
s2512228@icho01:~/Programming/programming report2$ ./s2512228-1.exe
alpha
beta
gamma
...
001 alpha
002 beta
003 gamma
s2512228@icho01:~/Programming/programming report2$
```

図 1 設問 1 の実行結果

図 1 より、入力した 3 行が 001 から始まる 3 桁の行番号付きで正しく表示されることを確認した。

また、100 行の文字列を入力し、最後まで正しく保存・表示できるかを確認した。実行結果の末尾を図 2 に示す。

```
097 97
098 98
099 99
100 100
s2512228@icho01:~/Programming/programming report2$
```

図 2 100 行入力した場合の実行結果の末尾

図 2 より、行番号 097 から 100 までが正しく 3 桁で表示され、100 行目まで処理した後にプログラムが終了することを確認した。以上より、終了条件、最大個数、行番号の書式、動的メモリの確保と解放が設問の条件を満たしていると判断した。

3 設問 2

3.1 設問内容の理解と実装方針

設問 2 では、設問 1 で保存した各行について、置換元の文字列 s が含まれていれば、その行で最初に現れる s だけを置換後の文字列 t に変更する。この操作を、 s または t の先頭が ... となるまで繰り返す。

`fgets` で s と t を読み込むと、通常は末尾に改行コードが含まれる。そのままでは入力行中の文字列と一致しにくいため、講義ノートの演習で扱われた方法に従い、末尾の改行コードを削除する関数 `chomp` を用意した。各行における最初の出現位置は `strstr` で求める。見つかった位置のポインタを関数 `substitution` に渡し、置換前部分、 t 、置換後部分の順に 1 文字ずつ新しい領域へコピーする。

新しい文字列に必要な大きさは、

$$\text{元の長さ} - s\text{の長さ} + t\text{の長さ} + 1$$

である。最後の 1 はヌル文字の領域である。新しい文字列の作成に成功したら、古い行の領域を `free` し、`lines[i]` を新しい文字列のアドレスへ入れ替える。結果を確認しやすくするため、各置換操作の後に全行を表示する。

3.2 作成したプログラムとポイント

作成したプログラムをリスト 2 に示す。

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define LINES 100
6  #define BSIZE 100
7
8  char* chomp(char *s){
9      size_t len;
10
11     len = strlen(s);
12     if(len > 0 && s[len - 1] == '\n'){
13         s[len - 1] = '\0';
14     }
15
16     return s;
17 }
18
19 void print_lines(char *lines[], int n){
20     int i;
21     size_t len;
22
23     for(i = 0; i < n; i++){
24         printf("%03d %s", i + 1, lines[i]);
25         len = strlen(lines[i]);
26         if(len == 0 || lines[i][len - 1] != '\n'){
27             printf("\n");
28         }
29     }
30 }
31
32 void free_lines(char *lines[], int n){
33     int i;
34
35     for(i = 0; i < n; i++){
36         free(lines[i]);
37     }
38 }
39
40 char* substitution(const char *old, const char *position,
41                    const char *s, const char *t){
42     size_t oldlen;
43     size_t slen;
44     size_t tlen;
45     size_t prefix;
46     size_t i;
47     size_t j;
48     size_t k;
49     char *x;
50
51     oldlen = strlen(old);
52     slen = strlen(s);
53     tlen = strlen(t);
54     prefix = (size_t)(position - old);
55
56     x = (char*)malloc(oldlen - slen + tlen + 1);
57     if(x == NULL){
58         return NULL;
```

```

59     }
60
61     k = 0;
62     for(i = 0; i < prefix; i++){
63         x[k] = old[i];
64         k++;
65     }
66
67     for(i = 0; i < tlen; i++){
68         x[k] = t[i];
69         k++;
70     }
71
72     j = prefix + slen;
73     while(j < oldlen){
74         x[k] = old[j];
75         j++;
76         k++;
77     }
78     x[k] = '\0';
79
80     return x;
81 }
82
83 int main(void){
84     char *lines[LINES];
85     char buf[BSIZE];
86     char s[BSIZE];
87     char t[BSIZE];
88     char *p;
89     char *x;
90     int n = 0;
91     int i;
92
93     while(n < LINES){
94         if(fgets(buf, BSIZE, stdin) == NULL){
95             break;
96         }
97
98         if(buf[0] == '.' && buf[1] == '.' && buf[2] == '.'){
99             break;
100         }
101
102         p = (char*)malloc(strlen(buf) + 1);
103         if(p == NULL){
104             printf("memory allocation failure.\n");
105             free_lines(lines, n);
106             exit(EXIT_FAILURE);
107         }
108
109         strcpy(p, buf);
110         lines[n] = p;
111         n++;
112     }
113
114     print_lines(lines, n);
115
116     while(1){
117         printf("source: ");
118         fgets(s, BSIZE, stdin);
119         chomp(s);
120         if(s[0] == '.' && s[1] == '.' && s[2] == '.'){

```

```

121         break;
122     }
123
124     printf("target: ");
125     fgets(t, BSIZE, stdin);
126     chomp(t);
127     if(t[0] == '.' && t[1] == '.' && t[2] == '.'){
128         break;
129     }
130
131     for(i = 0; i < n; i++){
132         p = strstr(lines[i], s);
133         if(p != NULL){
134             x = substitution(lines[i], p, s, t);
135             if(x == NULL){
136                 printf("memory allocation failure.\n");
137                 free_lines(lines, n);
138                 exit(EXIT_FAILURE);
139             }
140             free(lines[i]);
141             lines[i] = x;
142         }
143     }
144
145     printf("result:\n");
146     print_lines(lines, n);
147 }
148
149 free_lines(lines, n);
150
151 return 0;
152 }

```

Listing 2 設問2のプログラム (s2512228-2.c)

8-17 行目の `chomp` は、文字列の末尾が改行コードである場合に限り、その位置をヌル文字へ置き換える。19-30 行目の `print_lines` と 32-38 行目の `free_lines` は、表示処理と解放処理の重複を避けるための関数である。いずれの関数も配列と要素数を引数で受け取るため、大域変数は使用していない。

40-81 行目が置換後の文字列を作る `substitution` である。この関数内で呼び出している関数は、文字列長を求める `strlen` と領域を確保する `malloc` だけである。54 行目の `position-old` は、元の文字列の先頭から置換位置までの文字数を表す。56 行目で必要量を正確に計算して確保し、62-65 行目で置換位置より前、67-70 行目で `t`、72-77 行目で `s` より後の部分をコピーする。78 行目でヌル文字を置くことで、新しい文字列を正しく終端している。

116-147 行目が置換を繰り返す処理である。119 行目と 126 行目で改行を削除した後、それぞれの先頭が ... であるか確認する。132 行目の `strstr` が返すのは最初に一致した位置であるため、同じ行に `s` が複数存在しても最初の 1 か所だけが置換される。134 行目で新しい文字列を作り、140 行目で古い領域を解放してから 141 行目でアドレスを入れ替える。この順序により、元の文字列を破壊せずに新しい文字列を作成でき、使用しなくなった領域も残らない。

3.3 確認方法と結果

設問 1 と同様に、次のコマンドでコンパイルした。

```
gcc -std=c89 -o s2512228-2.exe s2512228-2.c
```

確認では、3 行を保存した後、`pars` を `lex` に置換した。続いて、存在しない `zzz` を `yyy` に置換しよ

うとした後、... を入力して終了した。主要部分の実行結果を次に示す。

```
s2512228@icho01:~/Programming/programming report2$ ./s2512228-2.exe
This is a parser.
parser parser
no match
...
001 This is a parser.
002 parser parser
003 no match
source: pars
target: lex
result:
001 This is a lexer.
002 lexer parser
003 no match
source: zzz
target: yyy
result:
001 This is a lexer.
002 lexer parser
003 no match
source: ...
s2512228@icho01:~/Programming/programming report2$
```

図 3 設問 2 における文字列置換の実行結果

1 行目では parser の pars が lex に置き換わり、lexer となった。2 行目には pars が 2 回含まれているが、最初のものだけが置換されて lexer parser となった。また、zzz はどの行にも存在しないため、その後の一覧は変化しなかった。

さらに、置換文字列の長さに関する条件を表 1 のように確認した。

表 1 設問 2 の追加確認

確認項目	入力例	確認結果
短い文字列への置換	abc → X	abc abc は X abc となり、最初の 1 か所だけが置換された。
長い文字列への置換	X → longertext	新しい長さに合わせて領域が確保され、文字列の欠落なく表示された。
空文字列への置換	short → 空文字列	該当部分が削除され、空の行として表示された。

```
s2512228@icho01:~/Programming/programming report2$ ./s2512228-2.exe
abc abc
...
001 abc abc
source: abc
target: X
result:
001 X abc
```

図 4 短い文字列へ置換した場合の実行結果

```
s2512228@icho01:~/Programming/programming report2$ ./s2512228-2.exe
X
...
001 X
source: X
target: longertext
result:
001 longertext
```

図 5 長い文字列へ置換した場合の実行結果

```
s2512228@icho01:~/Programming/programming_report2$ ./s2512228-2.exe
short
...
001 short
source: short
target:
result:
001
```

図 6 空文字列へ置換した場合の実行結果

置換前後で文字列の長さが変わる場合にも、必要なメモリ量を式で計算しているため、固定長配列の大きさに依存せずに処理できた。また、置換が行われた行では古い領域を解放し、プログラム終了時には残った全行を解放している。以上より、最初の一致だけを置換する条件、終了条件、メモリ確保の条件を満たしていると判断した。

4 感想

今回の課題では、ポインタの意味、仕組みを深く理解し [?], 文字列を単なる文字の並びとして扱うだけでなく、その文字列がメモリ上のどこに保存され、ポインタ配列には何が入っているのかを意識する必要があった。特に、`lines` に文字列そのものが入るのではなく、`malloc` で確保した領域の先頭アドレスが入るという点を意識してプログラムを組むと設問全体の処理が理解しやすくなった。

設問 2 では、元の領域を直接書き換えようとすると、置換前後の文字数が異なる場合に領域の不足や不要部分が生じてしまう。そこで、新しい長さを先に計算して別の領域を確保し、完成後に古い領域と入れ替える方法を用いた。この方法を考える中で文字列操作と動的メモリ管理が別々の内容でなく、密接に関係しているということを理解した。また、`fgets` が改行コードを含むことを忘れると探索が正しく行われなため、入力されたデータの実際の中身の確認に対する重要性を非常に感じた。今後は、正常な入力だけでなく、最大個数、該当なし、空文字列などの境界条件を先に整理してからプログラムを作成し、メモリの確保と解放 [?] を対応させることを意識したい。